Resilience Academy

SDI System Installation & Customization with CRD and KoboToolbox

Session 1.4: DevOps — Git, Docker, Docker Compose

Version Control & Container Orchestration for SDI

Masoud Hamad (SUZA) | **Msilikale Msilanga** (UTU)

Day 1 | Lecture 4 (45 min) | crd.resilienceacademy.ac.tz

Resilience Academy

Session Overview

Goal: Master the DevOps tools that power every SDI deployment

By end of this session you can explain the Git workflow, Docker concepts, and how Docker Compose orchestrates the full CRD stack.

Topics

- Git: clone, add, commit, push, pull
- Docker: images, containers, volumes, ports
- Docker Compose: multi-container YAML
- CRD stack orchestration

Activities

- Live terminal code-along
- Command reference cheat sheet
- Preview: CRD quick start (Day 2)

Git: Version Control Fundamentals

What is Git?

- Distributed **version control system**
- Tracks every change to every file
- Enables team collaboration on code
- CRD project is hosted on GitHub
- You `git clone` to get the code

Git Workflow

Essential Git Commands

Command	What It Does
`git clone <url>`	Download a reposi
`git status`	Show changed files
`git add .`	Stage all changes
`git commit -m "msg"`	Save a snapshot
`git push origin main`	Upload to GitHub
`git pull`	Download latest ch
`git log -oneline`	View commit histo
`git branch`	List branches
`git diff`	Show uncommitted

Git: Live Terminal Demo

Clone & Inspect GeoNode Project

```
# Clone the GeoNode project
git clone https://github.com/GeoNode/
  geonode-project.git my-crd
cd my-crd

# Inspect the repository
git log --oneline -10
git branch -a
git status

# View project structure
ls -la
cat README.md | head -20
```

Making Changes

```
# Create a new branch
git checkout -b my-customisation

# Edit a file
nano .env

# Stage and commit
git add .env
git status
git commit -m "Configure CRD settings"

# View your commit
git log --oneline -3
```

Day 2: You will clone and configure the full CRD project

```
git clone https://github.com/GeoNode/geonode-project.git
```

Docker: Core Concepts

Images vs Containers

- **Image** = blueprint (read-only template)
  - Like a recipe or class definition
  - Downloaded from Docker Hub
- **Container** = running instance of an image
  - Like a cooked meal or object instance
  - Has its own filesystem, network, processes

Volumes

- Persistent storage for container data
- Data survives container restart/removal
- CRD uses volumes for: database, uploaded layers, GeoServer data

Port Mapping

- Map host port → container port
- `-p 80:8000` means:
  host port 80 → container port 8000
- Allows access from your browser

CRD Port Mappings

80→GeoNode	8080→GeoServer	5432→PostGIS

Docker: Essential Commands

Docker Basics

```
# Verify installation
docker --version
docker compose version

# Test Docker works
docker run hello-world

# Pull an image
docker pull nginx:latest

# Run a container
docker run -d --name web \
  -p 8080:80 nginx:latest

# Access: http://localhost:8080
```

Container Management

```
# List running containers
docker ps

# View logs
docker logs -f web

# Enter a container shell
docker exec -it web bash

# Stop and remove
docker stop web
docker rm web

# List images
docker images

# System resource usage
docker stats
```

Docker Compose: Multi-Container Orchestration

What is Docker Compose?

- Define **multiple services** in one YAML file
- Start everything with **one command**
- Manages networks, volumes, dependencies
- CRD uses 6+ containers orchestrated by Compose

CRD Compose Services

Service	Image
django	GeoNode application
geoserver	OGC map server
db	PostgreSQL + PostGIS

Compose Commands

Command	Action
`up -d`	Start all services
`down`	Stop and remove
`ps`	List service status
`logs -f`	Follow all logs
`logs -f django`	Follow one service
`exec django bash`	Shell into service
`build`	Build images
`pull`	Update images
`restart`	Restart services

One Command to Rule Them All

Docker Compose: YAML Structure

docker-compose.yml (simplified)

```
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    depends_on:
      - app

  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgres://...
    depends_on:
      - db
```

YAML Key Concepts

- **services:** — each container definition
- **image:** — Docker Hub image to use
- **build:** — build from local Dockerfile
- **ports:** — host:container port mapping
- **environment:** — env variables
- **depends_on:** — startup order
- **volumes:** — persistent data storage

CRD Quick Start (Day 2)

1. `git clone geonode-project`
2. `cp .env.sample .env`
3. Edit configuration

Essential Linux Commands for SDI

Files & Navigation

```
pwd # where am I?
ls -la # list files
cd /path # change directory
cd ~ # go home
mkdir myproject # create directory
touch file.txt # create file
cp src dst # copy
mv src dst # move / rename
rm file.txt # delete
cat file.txt # view file
nano file.txt # edit file
sudo command # run as superuser
```

System & Networking

```
top # process monitor
free -h # memory usage
df -h # disk usage
uname -a # system info
ip addr show # network interfaces
ss -tlnp # listening ports

# Query live CRD API
curl "https://crd.resilience
  academy.ac.tz/api/v2/
  datasets/?search=mwanza" \
  | python3 -m json.tool \
  | head -20
```

Command Reference Cheat Sheet

Git

```
git clone <url>
git status
git add .
git commit -m "msg"
git push origin main
git pull
git log -oneline
```

Docker

```
docker run hello-world
docker ps
docker logs -f <name>
docker exec -it <n> bash
docker stop <name>
docker rm <name>
docker images
```

Docker Compose

```
docker compose up -d
docker compose down
docker compose ps
docker compose logs -f
docker compose exec
docker compose build
docker compose pull
```

Verification Checkpoint

Run these commands and confirm they all work:

`git -version` `docker -version` `docker compose version` `python3 -version`

Session 1.4 Summary & Day 1 Wrap-Up

Session 1.4 Takeaways

- ✓ Git: clone, add, commit, push workflow
- ✓ Docker: images, containers, volumes, ports
- ✓ Docker Compose: multi-container YAML orchestration
- ✓ Linux essentials for server management
- ✓ CRD starts with one command: `docker compose up -d`

Day 1 Achieved

- ✓ SDI concepts & CRD platform understood
- ✓ Software stack reviewed

Afternoon Practicals

- **Practical 1.1:** Install dev environment (3h)
- **Practical 1.2:** Linux server navigation (3h)
- **Practical 1.3:** Git, Docker, Compose hands-on (3h)

Day 2 Preview: CRD Installation

- GeoNode architecture deep-dive
- CRD Docker Compose installation
- CRD Ubuntu bare-metal installation
- Customise branding & settings
- Load Tanga & Mwanza datasets